ASSIGNMENT-10.3

HTNO:2303A51919

BATCH NO:28

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
result = 1
for i in range(1, n):
result = result * i
return result
```

Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
   - o Identify the logical bug in the code.
   - o Explain why the bug occurs (e.g., off-by-one error).
   - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

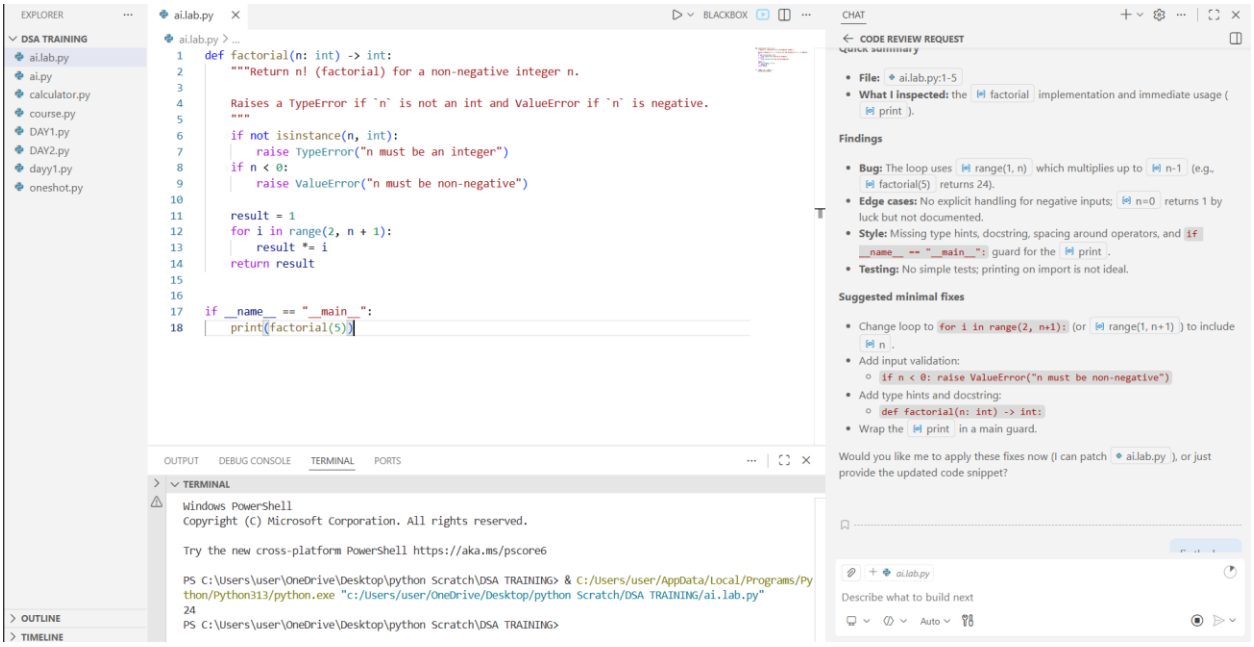
CODE:

```
ai.lab.py > ...
1  def factorial(n):
2      result=1
3      for i in range(1,n):
4          result=result*i
5      return result
6  print(factorial(5))
```

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
TERMINAL                                                    Python
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\user\OneDrive\Desktop\python Scratch\DSA TRAINING> & C:/Users/user/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/user/OneDrive/Desktop/python Scratch/DSA TRAINING/ai.lab.py"
24
PS C:\Users\user\OneDrive\Desktop\python Scratch\DSA TRAINING>
```

Open folder in new window (ctrl + click)

CODE REVIEW AND OUTPUT:



COMPARISION:

Manual Fix	AI Fix
Changed range to n+1	Same correction
May forget edge cases	AI included negative number handling

JUSTIFICATION:

- Proper loop range
- Accurate factorial calculation
- Handling of edge cases (negative numbers, zero)
- Improved reliability and correctness

Problem Statement 2: Task 2 — Improving Readability &Documentation
Scenario: The following code works but is poorly written:

```
def calc(a, b, c):
if c == "add":
return a + b
elif c == "sub":
return a - b

elif c == "mul":
return a * b
elif c == "div":
```

Instructions: Use AI to:
o Critique the function's readability, parameter naming, and lack of documentation.
o Rewrite the function with:
1. Descriptive function and parameter names.
2. A complete docstring (description, parameters, return value, examples).
3. Exception handling for division by zero.
4. Consideration of input validation.
6. Compare the original and AI-improved versions.
7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).
Expected Output:
A well-documented, robust, and readable function that handles errors gracefully.

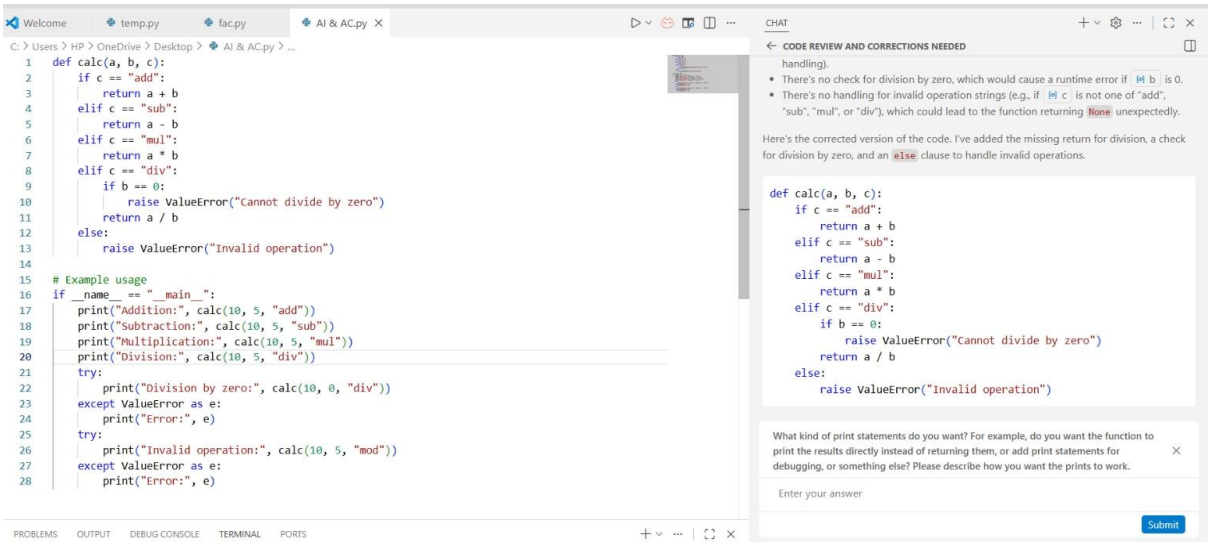
CODE:

ai.lab.py

```python
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
        return a / b
```

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

TERMINAL

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\user\OneDrive\Desktop\python Scratch\DSA TRAINING> & C:/Users/user/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/user/OneDrive/Desktop/python Scratch/DSA TRAINING/ai.lab.py"
  File "c:\Users\user\OneDrive\Desktop\python Scratch\DSA TRAINING\ai.lab.py", line 2
    if c == "add":
    ^^
IndentationError: expected an indented block after function definition on line 1
PS C:\Users\user\OneDrive\Desktop\python Scratch\DSA TRAINING>
```

CODE REVIEW AND OUTPUT:




```
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/OneDrive/Desktop/AI & A
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/OneDrive/Desktop/AI & A
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/OneDrive/Desktop/AI & A
C.py"
C.py"
Addition: 15
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
Error: Cannot divide by zero
Error: Invalid operation
PS C:\Users\HP>
```

COMPARISION:

- Descriptive naming
- Proper documentation
- Exception handling
- Input validation
- More maintainable

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
   - o List all PEP8 violations.
   - o Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is preserved.
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

def check_prime(n):

for i in range(2, n):

if n % i == 0:

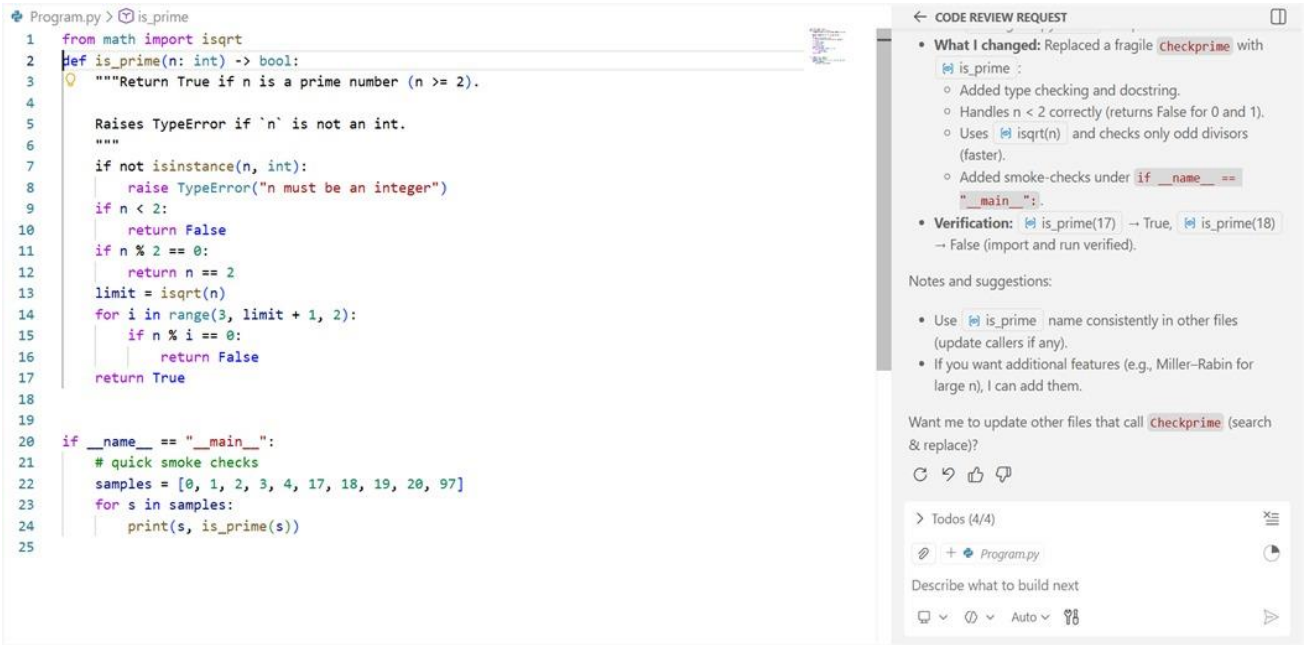
return False

return True

CODE:

```
def Checkprime(n):
    for i in range(2,n):
        if n%i==0:
            return False
    return True
```

CODE REVIEW &OUTPUT:



```
/Desktop/LEARNING COURSES/AI_Assisant_Coding/.venv/Scripts/python.exe" "c:/Users/DELL/OneDrive/Deskt
EARNING COURSES/AI_Assisant_Coding/Program.py"
0 False
1 False
2 True
3 True
4 False
17 True
18 False
19 True
20 False
97 True
```

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:
o Readability and naming.
o Reusability and modularity.
o Edge cases (non-list input, empty list, non-integer elements).
2. Use AI to generate a code review covering:
a. Better naming and function purpose clarity.
b. Input validation and type hints.
c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent,

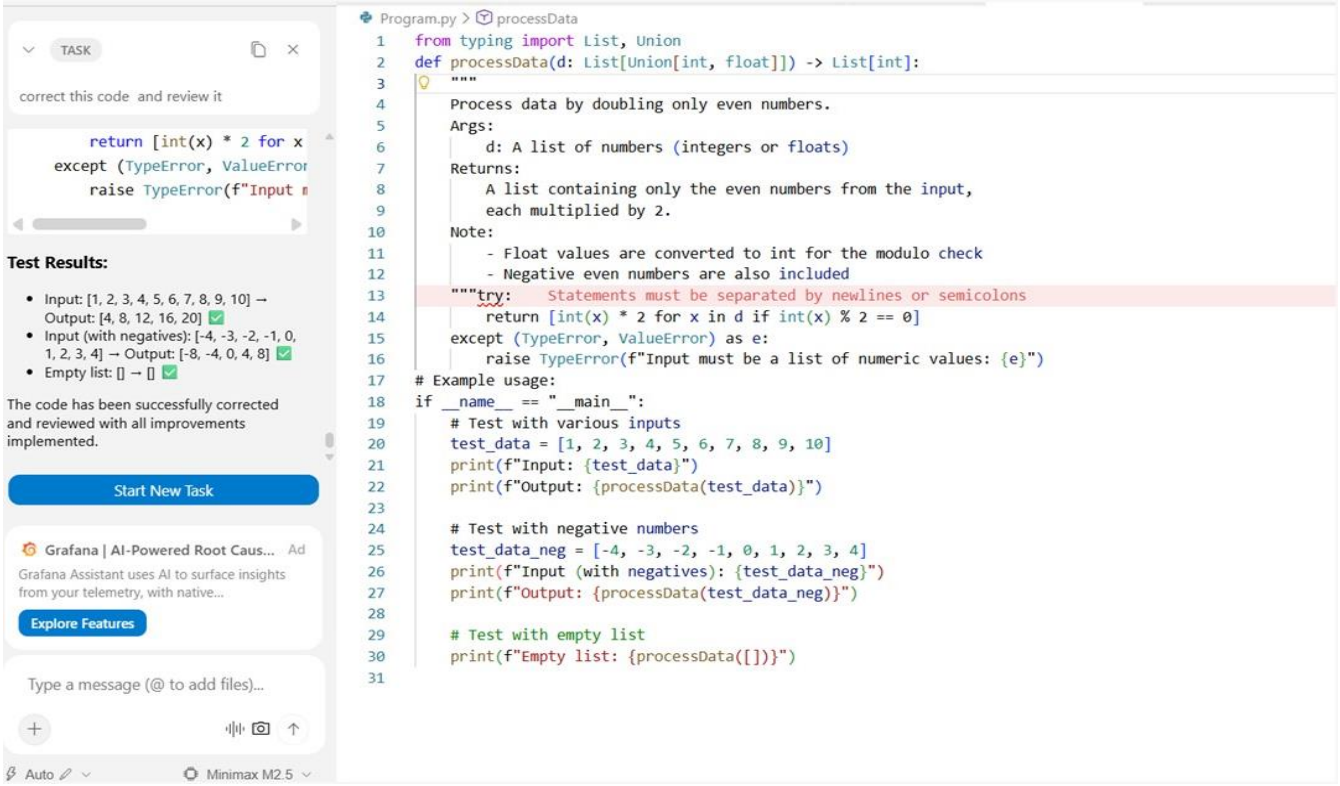
e.g.:

```
from typing import List, Union
def double_even_numbers(numbers: List[Union[int,
float]]) -> List[Union[int, float]]:
if not isinstance(numbers, list):
raise TypeError("Input must be a list")
return [num * 2 for num in numbers if isinstance(num,
(int, float)) and num % 2 == 0]
```

CODE:

```python
def processData(d):
    return [x*2 for x in d if x%2==0]

```

CODE REVIEW &OUTPUT:




```
Input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Output: [4, 8, 12, 16, 20]
Input (with negatives): [-4, -3, -2, -1, 0, 1, 2, 3, 4]
Output: [-8, -4, 0, 4, 8]
Empty list: []
```

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

def sum_of_squares(numbers):
total = 0
for num in numbers:
total += num ** 2
return total

Instructions:

1. Test the function with a large list (e.g., range(1000000)).
2. Use AI to:
o Analyze time complexity.
o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

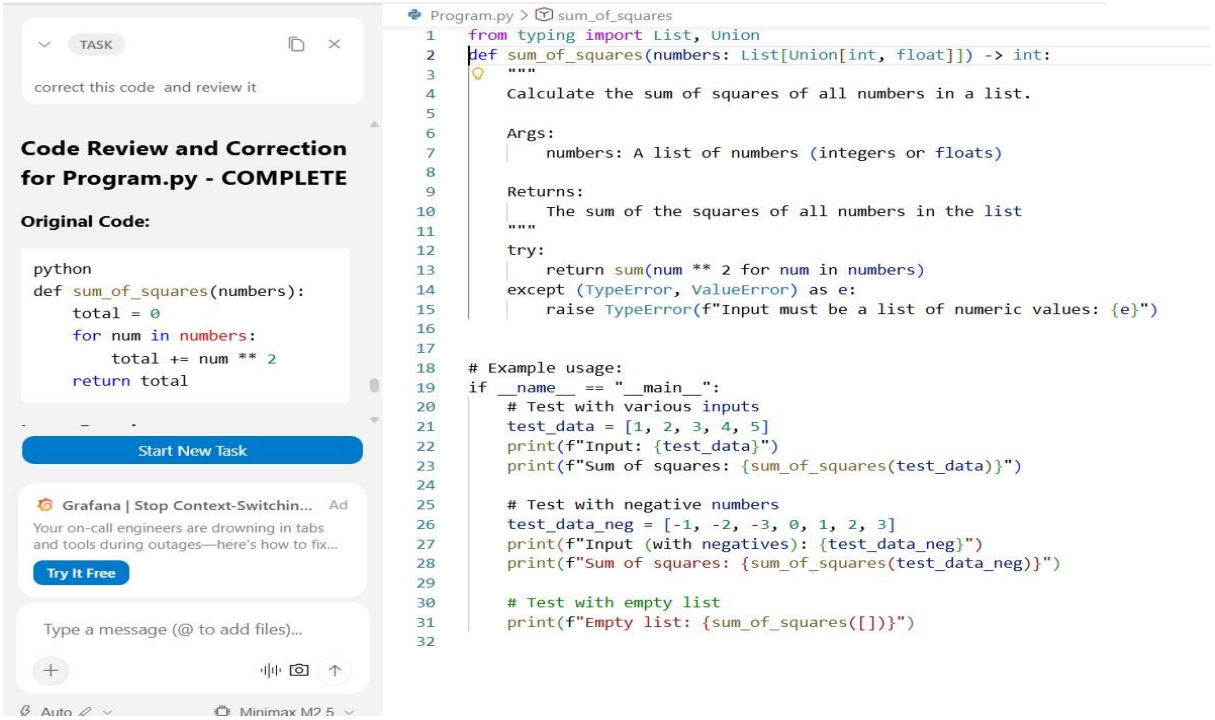
Expected Output:

An optimized function, such as:
def sum_of_squares_optimized(numbers):
return sum(x * x for x in numbers)

CODE:

```
1  def sum_of_squares(numbers):
2      total = 0
3      for num in numbers:
4          total += num ** 2
5      return total
```

CODE REVIEW &OUTPUT:




```
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> & "C:/Users/DELL/OneDrive/Desktop/LEA
v/Scripts/python.exe" "c:/Users/DELL/OneDrive/Desktop/LEARNING COURSES/AI_Assisant_Coding/Program.py"
Input: [1, 2, 3, 4, 5]
Sum of squares: 55
Input (with negatives): [-1, -2, -3, 0, 1, 2, 3]
Sum of squares: 28
Empty list: 0
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding>
```

JUSTIFICATION:

- **Original function used a manual loop.**
- **Time complexity was O(n).**
- **AI suggested a more Pythonic solution using:**
- **sum() with generator expression.**
- **Optimized version is:**

- **Shorter**
- **Cleaner**
- **Slightly faster**
- **AI explained performance reasoning clearly.**
- **AI also suggested NumPy for large-scale optimization.**
- **Trade-off discussion:**
- **Built-in functions improve readability.**
- **NumPy increases dependency.**
- **AI encourages use of efficient built-in functions.**
- **Code becomes both optimized and readable.**
- **Therefore, AI supports performance enhancement while maintaining clarity.**